

Resolución del Práctico 3

Testing de Software 2024

Índice

Introducción	3
1. Ejercicio 1	3
2. Ejercicio 2	3
2.1. Archivos trabajados	3
2.2. Modelo del Dominio de Entradas (MDE)	3
2.3. Requisitos de test para Pairwise Coverage (PWC)	3
2.4. Casos implementados	4
2.5. Implementación	5
2.6. Ejecución	5
3. Ejercicio 3	5
3.1. Archivos trabajados	5
3.2. Análisis del modelo original	5
3.3. MDE corregido	6
3.4. Base Choice (BC): requisitos y casos	6
3.5. Implementación	7
3.6. Ejecución	7
4. Ejercicio 4	7
4.1. Archivos trabajados	7
4.2. Modelo del Dominio de Entradas (MDE)	8
4.3. Requisitos pairwise y casos de prueba	8
4.4. Defectos observados en la versión original	8
4.5. Corrección documentada	8
4.6. Ejecución	9

5. Ejercicio 5	9
5.1. Referencia	9
5.2. Archivos trabajados	9
5.3. MDE utilizado	10
5.4. Cobertura y suite implementada	10
5.5. Resultado	10
5.6. Ejecución	10
Conclusiones	10

Introducción

Este documento presenta la resolución integral del *Práctico 3*. Se mantiene la línea de trabajo de los prácticos anteriores: redacción formal, organización por ejercicio y fundamentación de diseño de pruebas a partir de modelos del dominio de entradas y criterios de cobertura.

1. Ejercicio 1

Lectura del capítulo 6 (*Input Space Partitioning*) del libro *Introduction to Software Testing*.

2. Ejercicio 2

Se diseñó y ejecutó una batería de pruebas para `numberOfOccurrences(List<Integer>l, Integer element)` utilizando *Input Space Partitioning* y cobertura *pairwise*.

2.1. Archivos trabajados

- `src/main/java/assignment3_exercises/ListUtils.java`
- `src/test/java/assignment3_exercises/numberOfOccurrencesTest.java`

2.2. Modelo del Dominio de Entradas (MDE)

Características y bloques:

- C1 (referencia de l): L0 = `null`, L1 = no `null`.
- C2 (referencia de element): E0 = `null`, E1 = no `null`.
- C3 (tamaño de lista, si L1): S0 = vacía, S1 = no vacía.
- C4 (ocurrencias, si L1 y E1): O0 = 0, O1 = 1, O2 = más de 1.

Restricciones:

- Si L0, C3 y C4 no aplican.
- Si E0, C4 no aplica.
- Si S0, necesariamente O0.

2.3. Requisitos de test para Pairwise Coverage (PWC)

- R01: (L0, E0)

- R02: (L0, E1)
- R03: (L1, E0)
- R04: (L1, E1)
- R05: (L1, S0)
- R06: (L1, S1)
- R07: (L1, O0)
- R08: (L1, O1)
- R09: (L1, O2)
- R10: (E0, S0)
- R11: (E0, S1)
- R12: (E1, S0)
- R13: (E1, S1)
- R14: (E1, O0)
- R15: (E1, O1)
- R16: (E1, O2)
- R17: (S0, O0)
- R18: (S1, O0)
- R19: (S1, O1)
- R20: (S1, O2)

2.4. Casos implementados

- TC1: `l = null, element = null` $\rightarrow$ `IllegalArgumentException` (R01).
- TC2: `l = null, element = 7` $\rightarrow$ `IllegalArgumentException` (R02).
- TC3: `l = [], element = null` $\rightarrow$ `IllegalArgumentException` (R03, R05, R10).
- TC4: `l = [1,2,3], element = null` $\rightarrow$ `IllegalArgumentException` (R03, R06, R11).
- TC5: `l = [], element = 5` $\rightarrow$ 0 (R04, R05, R07, R12, R14, R17).

- TC6: $l = [1, 2, 3]$, $element = 9 \rightarrow 0$ (R04, R06, R07, R13, R14, R18).
- TC7: $l = [1, 2, 3]$, $element = 2 \rightarrow 1$ (R04, R06, R08, R13, R15, R19).
- TC8: $l = [4, 1, 4, 4]$, $element = 4 \rightarrow 3$ (R04, R06, R09, R13, R16, R20).

Además, se incorporó un caso de robustez para verificar que la lista de entrada no sea modificada por la rutina.

2.5. Implementación

La implementación en `ListUtils.numberOfOccurrences` valida precondiciones y luego contabiliza coincidencias con `element.equals(actual)`.

2.6. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Dtest=numberOfOccurrencesTest test
```

3. Ejercicio 3

Se analizó el MDE propuesto para `SetUtils.intersection(Set<Integer>set1, Set<Integer>set2)` y se diseñó una suite de pruebas con criterio *Base Choice*.

3.1. Archivos trabajados

- `src/main/java/assignment3_exercises/SetUtils.java`
- `src/test/java/assignment3_exercises/IntersectionTest.java`

3.2. Análisis del modelo original

a) Completitud de C1 (y análogamente C2)

Sí, es completa: para `set1` las opciones son `null`, vacío o no vacío, y esos casos quedan cubiertos.

b) No solapamiento de C1 (y análogamente C2)

Sí, no hay solapamiento: un set no puede ser simultáneamente `null`, vacío y no vacío.

c) Completitud de C3

No se cumple. Contraejemplo:

$$set1 = \{1, 2\}, \quad set2 = \{2, 3\}$$

La intersección no es vacía, pero no son iguales ni subconjuntos entre sí, y tampoco son disjuntos.

d) Disyunción de C3

Tampoco se cumple. Contraejemplo:

$$set1 = set2 = \{1, 2\}$$

Si “subconjunto” no se restringe a subconjunto propio, la entrada cae a la vez en “iguales” y en “set1 subconjunto de set2”.

3.3. MDE corregido

- C1: `set1 == null`, `set1 = {}`, `set1` no vacío.
- C2: `set2 == null`, `set2 = {}`, `set2` no vacío.
- C3 (solo si C1 y C2 no nulos):
 - C3.b1: `set1 = set2`
 - C3.b2: `set1` subconjunto propio de `set2`
 - C3.b3: `set2` subconjunto propio de `set1`
 - C3.b4: disjuntos
 - C3.b5: solapamiento parcial (intersección no vacía y sin inclusión)

Con esta revisión, C3 resulta completa y disyunta.

3.4. Base Choice (BC): requisitos y casos

Bloques base elegidos:

- C1 base: `set1` no vacío.
- C2 base: `set2` no vacío.
- C3 base: conjuntos iguales.

Requisitos BC considerados:

- TR-BC1: caso base.
- TR-BC2: variar C1 a `null` $\rightarrow$ `NullPointerException`.
- TR-BC3: variar C1 a vacío (ajustado por factibilidad).
- TR-BC4: variar C2 a `null` $\rightarrow$ `NullPointerException`.
- TR-BC5: variar C2 a vacío (ajustado por factibilidad).
- TR-BC6: variar C3 a “set1 subconjunto propio de set2”.
- TR-BC7: variar C3 a “set2 subconjunto propio de set1”.
- TR-BC8: variar C3 a disjuntos.
- TR-BC9: variar C3 a solapamiento parcial.

Los casos TC1 a TC9 de `IntersectionTest` cubren estos requisitos y agregan, además, validaciones de no mutación de entradas, nueva instancia de retorno y borde con ambos conjuntos vacíos.

3.5. Implementación

`SetUtils.intersection` se implementó para:

- lanzar `NullPointerException` ante entradas `null`,
- devolver un nuevo `Set` con la intersección,
- no modificar los conjuntos recibidos.

3.6. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Dtest=IntersectionTest test
```

4. Ejercicio 4

Se modeló y probó el método `PatternIndex.patternIndex(String subject, String pattern)` aplicando cobertura *pairwise* con restricciones de factibilidad.

4.1. Archivos trabajados

- `src/main/java/assignment3_exercises/PatternIndex.java`

- `src/test/java/assignment3_exercises/PatternIndexTest.java`

4.2. Modelo del Dominio de Entradas (MDE)

- C1: referencia de `subject` (`null` / no `null`).
- C2: referencia de `pattern` (`null` / no `null`).
- C3: tamaño de `subject` (vacío / no vacío), si C1 no es `null`.
- C4: tamaño de `pattern` (vacío / 1 carácter / 2 o más), si C2 no es `null`.
- C5: tipo de resultado (índice inicial / intermedio / final / -1), si C1 y C2 no son `null`.

Restricciones destacadas:

- Si C1 o C2 es `null`, se espera excepción y C3–C5 no aplican.
- Si `pattern` es vacía, el resultado esperado es 0.
- Si `subject` es vacía y `pattern` no vacía, el resultado esperado es -1.

4.3. Requisitos pairwise y casos de prueba

Se definieron 20 requisitos de cobertura por pares (R01–R20) y se implementaron 15 casos (TC1–TC15) que cubren:

- validación de precondiciones en entradas `null`,
- patrón vacío,
- coincidencia al inicio, medio y final,
- patrón no encontrado,
- sujeto vacío con patrón no vacío.

4.4. Defectos observados en la versión original

La batería de tests permite evidenciar dos problemas clásicos en la implementación base:

- ausencia de validación de `null` según especificación,
- manejo incorrecto del caso `pattern` vacía.

4.5. Corrección documentada

La corrección aplicada/esperada para satisfacer la especificación es:

```
if (subject == null || pattern == null) {
    throw new IllegalArgumentException(...);
}
if (pattern.isEmpty()) {
    return 0;
}
if (pattern.length() > subject.length()) {
    return -1;
}
// buscar primera ocurrencia y devolver su indice
```

4.6. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Dtest=PatternIndexTest test
```

5. Ejercicio 5

Se diseñaron pruebas sobre `Iterator` de `ArrayList` a partir de la especificación oficial de Java, aplicando modelado de características y cobertura de pares.

5.1. Referencia

- Oracle Java SE 7, `java.util.Iterator`: <https://docs.oracle.com/javase/7/docs/api/java/util/Iterator.html>

Puntos de especificación utilizados:

- `hasNext()` informa si existe próximo elemento.
- `next()` devuelve el siguiente elemento o lanza `NoSuchElementException`.
- `remove()` es opcional.
- `remove()` lanza `UnsupportedOperationException` si no está soportado.
- `remove()` lanza `IllegalStateException` si se viola su precondition de uso.

5.2. Archivos trabajados

- `src/test/java/assignment3_exercises/IteratorArrayListTest.java`

5.3. MDE utilizado

Características modeladas:

- C1: el iterador tiene más valores `{true, false}`.
- C2: `next()` retorna objeto no null `{true, false}`.
- C3: `remove()` soportado `{true, false}`.
- C4: restricción de `remove()` satisfecha `{true, false}`.

Decisiones de instanciación:

- C3=true: `ArrayList.iterator()`.
- C3=false: `Collections.unmodifiableList(...).iterator()`.

5.4. Cobertura y suite implementada

Se cubrieron 13 requisitos (R01–R13) mediante casos TC1 a TC13, contemplando:

- estados de `hasNext()` en iterador con datos, vacío y agotado,
- `next()` con retorno no nulo, retorno nulo y excepción por agotamiento,
- `remove()` no soportado,
- `remove()` con precondition inválida,
- `remove()` exitoso con y sin elementos remanentes.

5.5. Resultado

En los escenarios modelados no se identificaron inconsistencias respecto de la especificación de `Iterator`. La suite confirma el comportamiento esperado tanto en caminos válidos como en excepciones contractuales.

5.6. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Dtest=IteratorArrayListTest test
```

Conclusiones

La resolución del Práctico 3 consolidó el trabajo con *Input Space Partitioning* aplicado a problemas concretos de especificación, modelado y diseño de pruebas. El uso combinado de *Pairwise Coverage* y *Base Choice* permitió estructurar suites justificadas por

requisitos explícitos, reducir redundancia y mantener trazabilidad entre modelo, casos y comportamiento esperado.